CSE252: Unit IV Study Notes

Unsupervised Learning

School of Computer Science & Engineering

Session 2026–2027

Contents

1 Unit IV: Unsupervised Learning **2**

1.1 Clustering Algorithms . . . 2

1.1.1 1. K-Means Clustering . . . 2

1.1.2 Choosing K . . . 3

1.1.3 2. Hierarchical Clustering . . . 4

1.1.4 3. Density-Based Spatial Clustering (DBSCAN) . . . 4

1.2 Dimensionality Reduction and Principal Component Analysis (PCA) . . . 6

1.2.1 Principal Component Analysis (PCA) . . . 6

1.3 Pattern Discovery: Association Rule Mining & Apriori . . . 8

1.4 Unsupervised Applications in Robotics & Sensor Processing . . . 9

1.5 Unit IV Revision Summary . . . 10

1 Unit IV: Unsupervised Learning

Unsupervised learning analyzes unlabelled datasets $\mathbf{X} = \{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n\}$ to discover underlying clusters, latent manifold structures, or lower-dimensional representations without human label intervention.

Supervised vs Unsupervised — and the Hard Part

	Supervised (Unit III)	Unsupervised (Unit IV)
Given	Features X *and* labels y	Features X only
Goal	Learn $f : X \to Y$	Discover structure inside X
Evaluation	Compare against known answers	Internal measures + domain judgement
Hard part	Obtaining enough labelled data	Knowing whether the structure is real

The honest difficulty: without labels there is no accuracy score. Running K-Means with $K = 3$ and $K = 7$ yields two entirely different, entirely plausible answers, and no single number settles it.

Why not use unsupervised learning everywhere, given that labels are expensive? Because it cannot tell you what you want to know. Clustering will give you five groups of transactions; nothing tells you which group is "fraud". **Unsupervised learning finds STRUCTURE; only labels supply MEANING.**

The practical hybrid: cluster first to understand the data, label a small representative sample from each cluster, then train a supervised model.

1.1 Clustering Algorithms

1.1.1 1. K-Means Clustering

K-Means partitions n observations into K disjoint clusters $C = \{C_1, \ldots, C_K\}$, assigning each point to the cluster with the closest centroid μ_k.

Formula & Mathematical Relation: K-Means Objective Function (Within-Cluster Sum of Squares - WCSS)

$$J_{\text{K-Means}} = \sum_{k=1}^{K} \sum_{\mathbf{x}_i \in C_k} \|\mathbf{x}_i - \mu_k\|^2 \tag{1}$$

In scikit-learn this quantity is exposed as `inertia_`.

Exam Tip & Key takeaway: Why You Cannot Choose K by Minimising J

J decreases monotonically as K increases. At $K = n$, every point is its own cluster, sits exactly on its own centroid, and $J = 0$—a perfect score with zero insight.

This is the *same* trap as training error in Unit III: **the metric you optimise cannot also be the metric you use to select model complexity.** Hence the elbow method and silhouette score.

Assumption baked into the formula: Euclidean distance plus centroids-as-means implies K-Means expects roughly **spherical, similarly-sized** clusters. **Scaling is required** (distance-based).

Algorithm Steps (Lloyd's Algorithm):

1. **Initialization:** Select K initial centroids randomly or via K-Means++.

2. **Assignment Step:** Assign each data point $\mathbf{x}_i$ to nearest centroid:

$$C_k^{(t)} = \left\{ \mathbf{x}_i : \|\mathbf{x}_i - \mu_k^{(t)}\|^2 \leq \|\mathbf{x}_i - \mu_j^{(t)}\|^2 \quad \forall j \right\} \tag{2}$$

3. **Update Step:** Recompute cluster centroids:

$$\mu_k^{(t+1)} = \frac{1}{|C_k^{(t)}|} \sum_{\mathbf{x}_i \in C_k^{(t)}} \mathbf{x}_i \tag{3}$$

4. **Convergence:** Repeat assignment and update steps until assignments stop changing.

Common Pitfall & Misconception: Initialisation Sensitivity

K-Means starts from *random* centroids, so two runs on identical data can converge to **different local minima**—meaning your "customer segments" could change between runs, which is unacceptable commercially.
Two fixes: set `random_state` for reproducibility, and set `n_init` (scikit-learn runs the algorithm several times and keeps the lowest inertia).
K-Means++ is the modern default initialisation: pick the first centroid at random, then bias later choices toward points *far* from existing centroids. This spreads them out and improves both speed and quality.
Convergence guarantee, stated precisely: the algorithm always terminates (each step can only decrease J, and there are finitely many assignments), but it converges to a *local* minimum—"it always stops" and "it always finds the best answer" are different claims. Complexity is roughly $O(n \cdot K \cdot d \cdot \text{iterations})$, which is why K-Means scales well.

Worked Example — One Iteration of K-Means

1-D data: $2, 3, 6, 8, 12, 15$; $K = 2$; initial centroids $\mu_1 = 3$, $\mu_2 = 12$.
Assignment step (distance is just absolute difference):

Point 2:	$\lvert 2-3\rvert = 1$	vs $\lvert 2-12\rvert = 10$	$\to C_1$
Point 3:	$\lvert 3-3\rvert = 0$	vs $\lvert 3-12\rvert = 9$	$\to C_1$
Point 6:	$\lvert 6-3\rvert = 3$	vs $\lvert 6-12\rvert = 6$	$\to C_1$
Point 8:	$\lvert 8-3\rvert = 5$	vs $\lvert 8-12\rvert = 4$	$\to C_2$
Points 12, 15:	clearly nearer to 12		$\to C_2$

Update step: $\mu_1 = \frac{2+3+6}{3} \approx 3.67$, $\mu_2 = \frac{8+12+15}{3} \approx 11.67$.
Second iteration: point 6 gives $|6 - 3.67| = 2.33$ vs $|6 - 11.67| = 5.67$ (stays in C_1); point 8 gives $|8 - 3.67| = 4.33$ vs $|8 - 11.67| = 3.67$ (stays in C_2). Nothing moves—**converged**.
Note point 8: it went to C_2 by a margin of one. Different initial centroids could have placed it elsewhere and changed the final clustering—initialisation sensitivity, made concrete.

1.1.2 Choosing K

Formula & Mathematical Relation: Evaluating Optimal K: Silhouette Score

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)} \quad \in [-1, +1] \tag{4}$$

where a_i is the mean intra-cluster distance (how close point i is to its *own* cluster) and b_i is the mean nearest-cluster distance (how close it is to the nearest *other* cluster).

Exam Tip & Key takeaway: Elbow Method vs Silhouette Score

Elbow method: run K-Means for $K = 1 \dots 10$ and plot WCSS against K. The "elbow" marks where additional clusters stop buying meaningful error reduction. *Weakness:* the elbow is often ambiguous—real curves frequently have no sharp bend.
Silhouette: average s_i over all points, for each K; choose the highest. This produces a defensible *number* rather than a bend requiring interpretation.

- $s \approx +1$: well clustered. $s \approx 0$: on a boundary. $s < 0$: probably misassigned.

The realistic situation: elbow says $K = 3$, silhouette says 5, and the business is staffed for 4 campaigns. There is no purely technical resolution—both methods are heuristics and the business constraint is a legitimate input. Present the options with their scores, and document the decision. **Unsupervised learning produces candidate structures; choosing among them requires judgement from outside the data.**

1.1.3 2. Hierarchical Clustering

Builds a multi-level tree of clusters called a **Dendrogram**. Its key advantage: **you do not have to choose K in advance.**

- **Agglomerative (Bottom-Up):** Starts with N individual clusters and merges the most similar cluster pair iteratively. (*Divisive* is top-down and less common.)
- **Linkage Criteria:**
  - *Single Linkage:* Minimum pair-wise distance $d(A, B) = \min\{d(\mathbf{a}, \mathbf{b})\}$. Prone to **chaining**—two separated blobs joined by a thin bridge of points get merged.
  - *Complete Linkage:* Maximum pair-wise distance $d(A, B) = \max\{d(\mathbf{a}, \mathbf{b})\}$. Produces compact clusters; may break up genuinely elongated ones.
  - *Average Linkage:* Mean distance across all cross-cluster pairs.
  - *Ward's Linkage:* Minimizes total within-cluster variance increase. **Usually the best default.**

Exam Tip & Key takeaway: Reading a Dendrogram

Leaves at the bottom are individual data points. Each horizontal join is a **merge**, and the **height** of that join is the *distance* at which the merge occurred.
To obtain clusters, **cut the dendrogram horizontally**; the number of vertical lines cut gives the number of clusters. Cutting *lower* yields *more* clusters (early in the merging process, when many small clusters still exist).
A **tall vertical line** before a merge indicates the two clusters were very dissimilar—a good place to cut.
Cost: roughly $O(n^2)$ memory and $O(n^3)$ time. A 100,000-point dataset would need a 10-billion-entry distance matrix, so use K-Means at that scale. Hierarchical clustering shines in bioinformatics, where the *tree itself* is the scientific output.

1.1.4 3. Density-Based Spatial Clustering (DBSCAN)

DBSCAN clusters points based on local density radius (ϵ) and minimum neighbor count (**MinPts**). No K is required—the number of clusters emerges from the data.

DBSCAN Point Classifications

- **Core Point:** Has $\geq$ MinPts within its ϵ-neighborhood.
- **Border Point:** Has $<$ MinPts in neighborhood, but falls within ϵ-radius of a Core Point.
- **Noise Point:** Neither Core nor Border point (flagged as outlier/anomaly, belonging to *no* cluster).

DBSCAN is the only algorithm in this unit that **explicitly identifies noise**.

Exam Tip & Key takeaway: K-Means vs. DBSCAN — the Concentric Rings Test

Sketch two **concentric rings** of points. K-Means with $K = 2$ slices them down the middle, producing two half-moon clusters cutting across *both* rings—completely wrong, because K-Means can only produce **convex** regions. DBSCAN follows the density and correctly recovers the inner and outer rings.

	K-Means	DBSCAN
Need K in advance?	Yes	No (but needs ϵ, MinPts)
Cluster shapes	Spherical, similar sizes	Arbitrary — rings, crescents
Handles outliers?	No (forces every point in)	**Yes** (labels them noise)
Scales to large n?	Yes	Reasonably

Generalisable lesson: when an algorithm fails, first ask whether you have violated one of its *assumptions*. If so, no amount of parameter tuning helps—you need a different algorithm.

Common Pitfall & Misconception: DBSCAN's Main Weakness

DBSCAN fails when clusters have **very different densities**, because a single ϵ must serve every cluster. If one cluster is tightly packed and another diffuse, no single ϵ works—you either merge the tight ones or fragment the diffuse one. **HDBSCAN** is the modern extension handling varying density.
It also struggles in high dimensions (where density loses meaning), and border points can be assigned differently depending on processing order.
Parameter heuristics: MinPts $\approx 2 \times$ dimensions; choose ϵ from the "knee" of a sorted k-distance plot.

Three Algorithms, Three Definitions of "Cluster"

- **K-Means:** a cluster is a set of points near a common centre.
- **Hierarchical:** a cluster is whatever survives repeated merging under your linkage rule.
- **DBSCAN:** a cluster is a connected dense region.

There is no universally correct definition, hence no universally correct algorithm.

Real-World Industry Application: E-Commerce Customer Market Segmentation

Online platforms group millions of registered users into $K = 5$ distinct purchasing behaviour personas based on Annual Spend (x_1), Transaction Frequency (x_2), and Return Rate (x_3)—for example: high-spend loyalists, frequent low-value buyers, occasional big spenders, serial returners, and dormant accounts.

The critical step: the algorithm produced five *numbered* groups. A **human** inspected `cluster_centers_` and assigned meaning. Clustering produces GROUPS, not MEANING.

How to validate a label: profile the cluster on features *not used* in the clustering. If "serial returners" also contact support more often, the label is capturing something real.

Caveats: scaling is essential (spend is in tens of thousands, return rate is a fraction); and re-running next quarter shifts both the boundaries and the cluster numbering—clustering is a snapshot, not a permanent taxonomy.

1.2 Dimensionality Reduction and Principal Component Analysis (PCA)

High-dimensional data suffers from the **Curse of Dimensionality**.

Common Pitfall & Misconception: The Curse of Dimensionality

- **Data becomes sparse.** 100 points along a line of length 10 are densely packed. In a 10×10 square they average one per unit square. In a $10 \times 10 \times 10$ cube most cells are empty. In four dimensions, 100 points occupy 10,000 cells. *The data has not changed; the space exploded around it.*
- **Distances stop discriminating.** As dimensions increase, the distances to your nearest and farthest neighbours converge—which breaks K-NN, K-Means and DBSCAN alike.
- **Overfitting becomes easier** (more parameters, more capacity to memorise noise—Unit III).
- **Everything gets slower**, and visualisation is impossible beyond three dimensions.

The key distinction: feature **selection** (Unit II) keeps a subset of your *original* columns and stays interpretable. **PCA** creates *new* features that are weighted mixtures of all originals—more powerful, but you lose the ability to name your features.

1.2.1 Principal Component Analysis (PCA)

PCA is an unsupervised linear technique that projects p-dimensional data onto k orthogonal directions (Principal Components) that maximize variance.

The Geometric Intuition

Picture an elongated cloud of points tilted at 45°. The x and y axes are not the natural description—the cloud's *own* long axis is. PCA finds that long axis (PC1), then the perpendicular short axis (PC2). Having rotated to those axes, you can drop PC2 and lose very little information.

Formula & Mathematical Relation: Mathematical Derivation of PCA

1. Mean-center data: $\mathbf{X}_{\text{centered}} = \mathbf{X} - \mu$. *(Without this, PC1 points at the mean rather than along the spread.)*
2. Compute Covariance Matrix $\mathbf{\Sigma} = \frac{1}{n-1}\mathbf{X}_{\text{centered}}^T\mathbf{X}_{\text{centered}}$. *(Off-diagonal entries measure tilt.)*
3. Solve Eigenvalue Problem $\mathbf{\Sigma}\mathbf{v}_i = \lambda_i\mathbf{v}_i$ where $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_p$. *(Eigenvectors are DIRECTIONS; eigenvalues are the VARIANCE along them.)*
4. Project data onto top k eigenvectors: $\mathbf{X}_{\text{pca}} = \mathbf{X}_{\text{centered}}\mathbf{W}_k$.

Explained Variance Ratio for component i:

$$\text{EVR}_i = \frac{\lambda_i}{\sum_{j=1}^{p}\lambda_j} \tag{5}$$

Components are **orthogonal**, therefore uncorrelated by construction.

Common Pitfall & Misconception: Always Standardise Before PCA

PCA maximises **variance**, and variance depends on **units**. A salary column in rupees has enormous variance relative to an age column, so PC1 would essentially *be* "salary" regardless of the underlying structure. Apply `StandardScaler` first, always.

Worked Example — Choosing k

Eigenvalues: 4.0, 2.0, 1.2, 0.5, 0.3; sum = 8.0.

$$\text{EVR} = 0.50,\ 0.25,\ 0.15,\ 0.0625,\ 0.0375$$
$$\text{Cumulative} = 0.50,\ 0.75,\ \mathbf{0.90},\ 0.9625,\ 1.00$$

To retain 90% of the variance, keep $\mathbf{k = 3}$.
Practical rules: keep the smallest k reaching 90–95% cumulative EVR; or plot EVR against component number and look for the elbow (a **scree plot**—the same visual heuristic as the elbow method for K). For visualisation use $k = 2$ or 3. In scikit-learn, `PCA(n_components=0.95)` selects k automatically. EVR values always sum to 1, which is a useful sanity check.

Exam Tip & Key takeaway: What PCA Costs You — and Why It Ignores y

Benefits: fights the curse of dimensionality; removes multicollinearity (components are orthogonal); faster training; enables 2-D visualisation; can reduce noise.
Costs: interpretability is lost ("PC1 = 0.4 age +0.7 income −0.2 tenure" means nothing to a domain expert); some information is discarded by design; it assumes *linearity* (curved structure needs kernel PCA, t-SNE or UMAP).
The subtle point: variance is not importance. PCA is *unsupervised*—it never looks at y. If a high-variance feature is measurement noise while a low-variance feature is genuinely predictive, PCA keeps the noise and may discard the signal.
The supervised alternative: Linear Discriminant Analysis (LDA) finds directions that best *separate the classes*. One-sentence exam answer: *PCA maximises variance; LDA maximises class separation.*
Consequence for high-stakes work: "your value on principal component 2 was too low" is not an acceptable explanation to a regulator. Prefer feature *selection* when interpretability is required.

1.3 Pattern Discovery: Association Rule Mining & Apriori

Discovers relational rules ($A \Rightarrow B$) in transactional databases T. Note this is a different kind of unsupervised learning: clustering groups *rows*; association rule mining finds relationships between *items* across transactions.

Formula & Mathematical Relation: Association Rule Metrics

1. **Support:** $\text{Support}(A \Rightarrow B) = \frac{\text{Count}(A \cup B)}{N}$ — *how often does this combination occur at all?* A popularity filter.
2. **Confidence:** $\text{Confidence}(A \Rightarrow B) = \frac{\text{Support}(A \cup B)}{\text{Support}(A)}$ — *when A happens, how often does B follow?* This is $P(B \mid A)$.
3. **Lift:** $\text{Lift}(A \Rightarrow B) = \frac{\text{Support}(A \cup B)}{\text{Support}(A) \times \text{Support}(B)}$ — *does A actually improve the odds of B?* Lift > 1 indicates positive association; $= 1$ independence; < 1 means A makes B *less* likely.

Exam Tip & Key takeaway: Confidence is Asymmetric; Lift is Symmetric

$\text{Confidence}(A \Rightarrow B) \neq \text{Confidence}(B \Rightarrow A)$ in general, because the denominators differ. But $\text{Lift}(A \Rightarrow B) = \text{Lift}(B \Rightarrow A)$—the formula is unchanged by swapping.
Terminology: an **itemset** is a set of items occurring together; a rule has an **antecedent** (A, left) and a **consequent** (B, right).

Worked Example — Support, Confidence and Lift

1,000 transactions. Bread appears in 400; Butter in 300; both together in 240. Evaluate Bread $\Rightarrow$ Butter.

$$\begin{aligned} \text{Support}(\text{Bread}) &= 400/1000 = 0.40 \\ \text{Support}(\text{Butter}) &= 300/1000 = 0.30 \\ \text{Support}(\text{Bread} \cup \text{Butter}) &= 240/1000 = 0.24 \\ \text{Confidence} &= 0.24/0.40 = 0.60 \\ \text{Lift} &= \frac{0.24}{0.40 \times 0.30} = \frac{0.24}{0.12} = 2.0 \end{aligned}$$

Lift $= 2.0$: bread buyers are *twice* as likely to buy butter as a random customer. A genuinely useful rule.
Now the critical insight. Suppose butter appeared in *900* of the 1,000 baskets. Then $\text{Lift} = \frac{0.24}{0.40 \times 0.90} = 0.67$—**below 1**. Bread buyers would be *less* likely than average to buy butter, while confidence remained a respectable-looking 0.60.
Lesson: high confidence with low lift means the consequent is simply popular, not associated. Always report lift. This is the same statistical trap as the accuracy paradox in Unit V—a metric ignoring the base rate misleads you.

The Apriori Property

"All non-empty subsets of a frequent itemset must also be frequent." Equivalently: **if an itemset is infrequent, every superset of it is also infrequent**, so all supersets can be pruned immediately without ever being counted.
Why it holds (the anti-monotone property): adding an item can only make an itemset *rarer*, never more common—a basket containing {Milk, Bread, Eggs} necessarily contains {Milk, Bread}. So support can only decrease as itemsets grow.
Scale of the saving: if {Milk, Bread} falls below minimum support, every superset containing both is eliminated—$2^{(p-2)}$ itemsets removed by a single failed count. With 10,000 products there are 2^{10000} possible itemsets, so this pruning is what makes the problem tractable at all.
The algorithm works level-wise: find frequent 1-itemsets, generate candidate 2-itemsets, count and prune, generate 3-itemsets from the survivors, and so on. Then form rules from each frequent itemset, keeping those meeting minimum confidence (and ideally lift > 1).
Cost: Apriori scans the database once per level. **FP-Growth** is a faster modern alternative avoiding repeated scans.

Real-World Industry Application: Supermarket Market Basket Analysis

A retail grocery chain discovers {Diapers} $\Rightarrow$ {Beer} with Support $= 0.04$, Confidence $= 0.72$, and Lift $= 2.8$. The chain reorganizes aisle layouts to place beer adjacent to baby products.
Reading the numbers: the combination occurs in 4% of all transactions; among diaper purchases, 72% also included beer; and the association is 2.8× stronger than chance.
Two cautions. First, this story is famous and its origins are *disputed*—treat it as an illustration, not a documented case. Second, the rule does *not* say nappies *cause* beer purchases; the "young parents" explanation is a hypothesis invented afterwards. The data supports the association, not the story.
Beyond retail: the same technique powers "customers who bought this also bought", medical symptom co-occurrence, machinery fault co-occurrence, and web clickstream analysis.

1.4 Unsupervised Applications in Robotics & Sensor Processing

1. **LiDAR 3D Point Cloud Segmentation:** Autonomous vehicles use DBSCAN Euclidean clustering to group 3D LiDAR point returns into discrete physical obstacles (pedestrians, vehicles, barriers). **Why DBSCAN:** the object count is unknown in advance, real objects produce irregular shapes, and LiDAR returns contain substantial noise from reflections and rain which DBSCAN labels rather than absorbing.

2. **Sensor Anomaly Detection:** Industrial turbines compress multi-axis vibration telemetry using PCA; the signal is then *reconstructed* from the retained components and the reconstruction error measured. Normal readings reconstruct almost perfectly because they lie in the space PCA learned; abnormal readings do not fit, so the error spikes—indicating bearing wear.

   Why this matters: it requires **no labelled failures.** You cannot train a supervised failure detector without examples of failure, and catastrophic failures are rare by design. Unsupervised methods sidestep this entirely.

3. **Robot Behaviour Discovery:** clustering logged sensor-actuator traces reveals recurring situations and operating modes nobody explicitly programmed—useful for diagnosing rare failure states across a fleet.

4. **Sensor Fusion Preprocessing:** robots carry many overlapping, correlated sensors; PCA removes the redundancy before data reaches the controller, reducing transmission and processing under real-time latency budgets.

All four share one property: **labelling is impractical at the required scale**—which is exactly when unsupervised methods earn their place.

```python
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans, DBSCAN, AgglomerativeClustering
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score
import numpy as np

# 0. ALWAYS scale first - every method here is distance/variance based
X_scaled = StandardScaler().fit_transform(X)

# 1. Choose K with the elbow method AND the silhouette score
for k in range(2, 8):
    km = KMeans(n_clusters=k, n_init=10, random_state=42)
    labels = km.fit_predict(X_scaled)
    print(k, round(km.inertia_, 1), round(silhouette_score(X_scaled, labels
   ), 3))

# 2. Fit the chosen K-Means; inspect centroids to NAME the clusters
kmeans = KMeans(n_clusters=5, n_init=10, random_state=42)
segments = kmeans.fit_predict(X_scaled)
print("Cluster centres:", kmeans.cluster_centers_)

# 3. DBSCAN - no K needed; label -1 means NOISE
db = DBSCAN(eps=0.5, min_samples=5).fit(X_scaled)
print("Noise points:", list(db.labels_).count(-1))

# 4. Hierarchical clustering with Ward linkage
agg = AgglomerativeClustering(n_clusters=3, linkage='ward').fit(X_scaled)

# 5. PCA - reduce to 2 components so clusters can be plotted
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
print("EVR:", pca.explained_variance_ratio_,
      "cumulative:", np.cumsum(pca.explained_variance_ratio_))
print("k for 95%:", PCA(n_components=0.95).fit(X_scaled).n_components_)
```

Listing 1: Scikit-Learn Clustering and PCA Workflow

1.5 Unit IV Revision Summary

1. **Unsupervised learning finds structure without labels.** No answer key means no accuracy score—judge with internal measures plus domain knowledge, and a human must supply the meaning.

2. **K-Means minimises WCSS** by alternating assign and update. You must choose K (elbow, silhouette); it assumes spherical clusters, is sensitive to initialisation, and needs scaled features.

3. **Hierarchical clustering builds a tree, not a partition.** No K needed in advance—cut the dendrogram where you like. Ward's linkage is the usual default; roughly $O(n^3)$, so it does not scale.

4. **DBSCAN clusters by density and names its outliers.** Core, border and noise points controlled by ϵ and MinPts; it finds arbitrary shapes that K-Means structurally cannot.

5. **PCA rotates the axes to where the variance is.** Centre, covariance, eigen-decompose, project. Keep enough components for 90–95% EVR. You buy compression and pay in interpretability.

6. **Association rules need all three metrics.** Support filters rarity, confidence gives $P(B|A)$, and **lift** tells you whether the association is real.